

Học sâu và ứng dụng

Giới thiệu keras và bài toán phân loại ảnh

Giới thiệu về keras

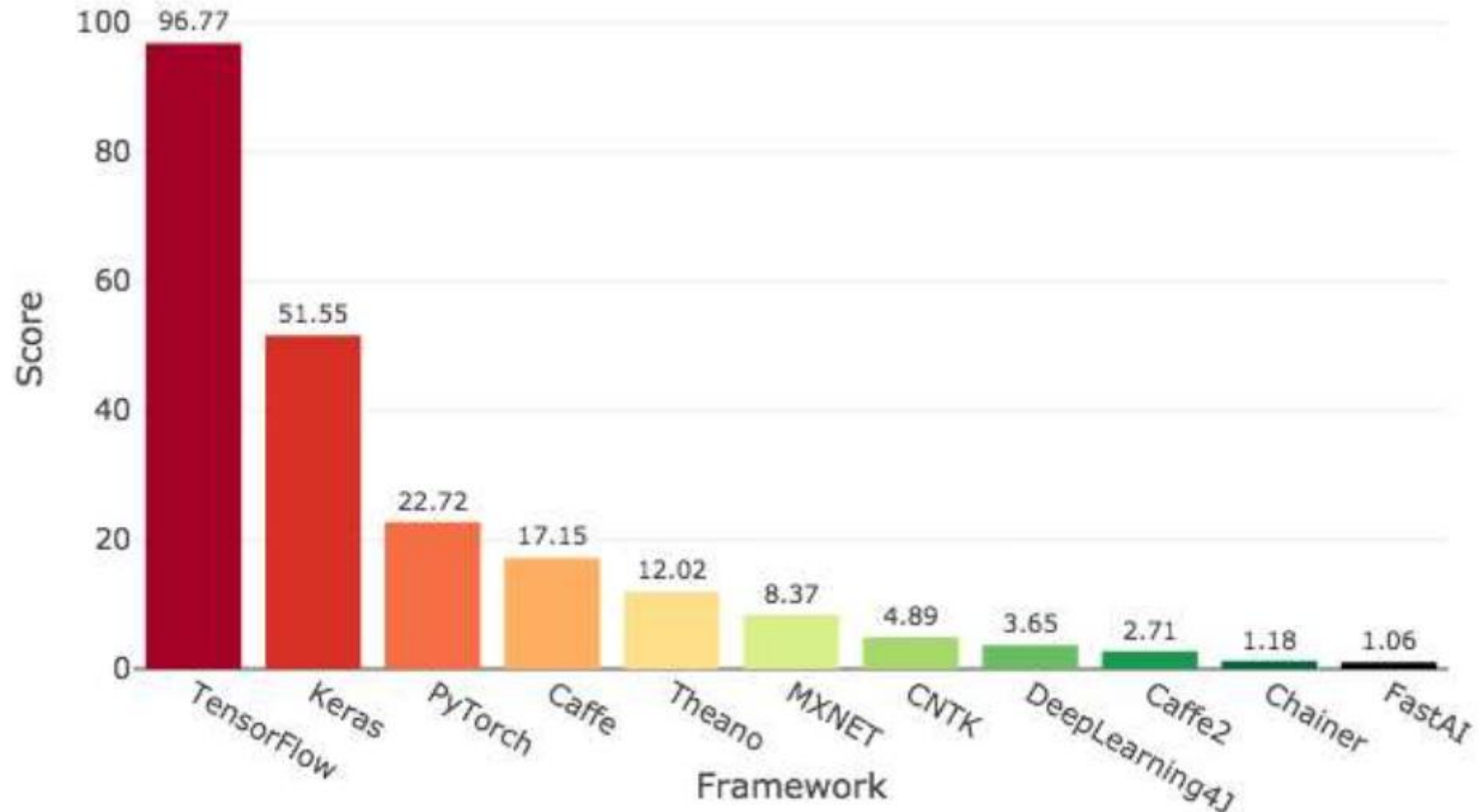
- **Bản chất của bài toán Deep learning:** Bạn có dữ liệu, bạn muốn máy tính học được các mô hình (model) từ dữ liệu, sau đó dùng mô hình đó để dự đoán được các dữ liệu mới. Các bước cơ bản làm một bài toán deep learning:
 1. Xây dựng bài toán
 2. Chuẩn bị dữ liệu (dataset)
 3. Xây dựng mô hình (model)
 4. Định nghĩa hàm mất mát (loss function)
 5. Thực hiện backpropagation và áp dụng gradient descent để tìm các tham số (parameter) gồm weight và bias để tối ưu loss function.
 6. Dự đoán dữ liệu mới bằng model với các hệ số tìm được ở trên

Giới thiệu về keras

- **Bản chất của bài toán Deep learning:**
 - Bước xây dựng model thì áp dụng các kiến thức được trình bày trong bài neural network và convolutional neural network ta có thể xây dựng model hoàn chỉnh từ đầu bằng python.
 - Tuy nhiên, bước backpropagation trở nên phức tạp hơn rất rất nhiều.
 - Khó để implement và tối ưu được tốc độ tính toán.
 - Đây là lý do các framework về deep learning ra đời với các đặc điểm:
 - Người dùng chỉ cần định nghĩa model và loss function, framework sẽ lo phần backpropagation.
 - Việc định nghĩa layer, activation function, loss function đơn giản hơn cho người dùng. Ví dụ để thêm layer trong neural network chỉ cần báo là layer có bao nhiêu node và dùng hàm activation gì.

Giới thiệu về keras

- Các deep learning framework phổ biến



Giới thiệu về keras

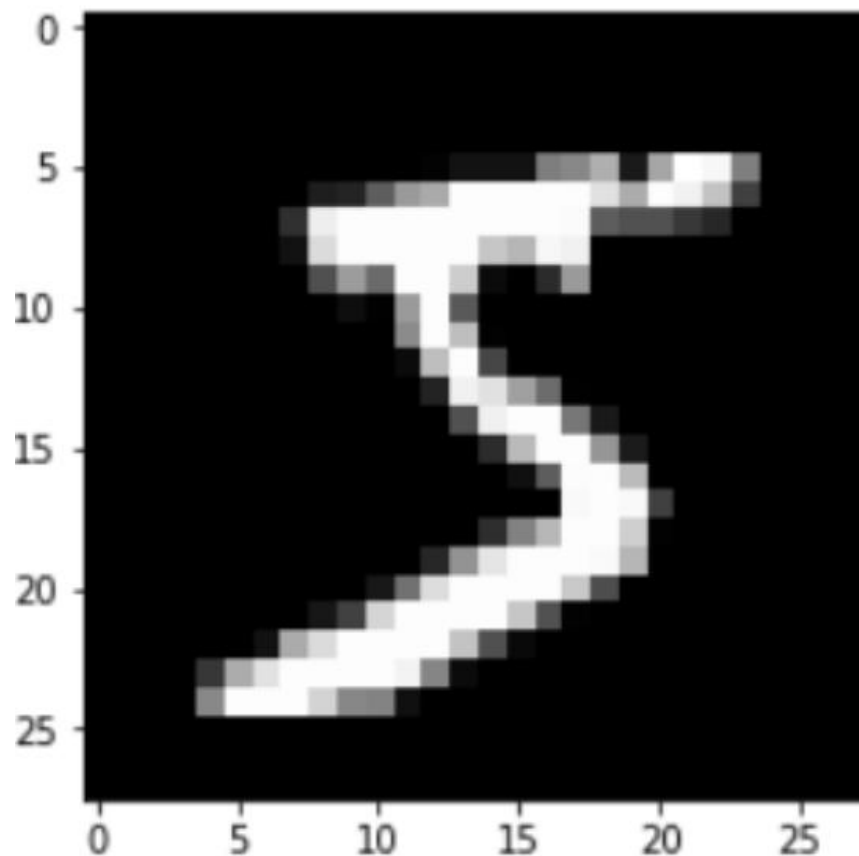
- Các deep learning framework phổ biến
 - Có thể thấy tensorflow là framework phổ biến nhất tuy nhiên tensorflow khá khó sử dụng cho người mới bắt đầu.
 - Về keras: dễ sử dụng, thân thiện với người dùng nhưng đủ tốt để làm các bài toán về deep learning.

Keras là một framework mã nguồn mở cho deep learning được viết bằng Python, có thể chạy trên nền của các deep learning framework khác như: tensorflow, theano, CNTK. Với các API bậc cao, dễ sử dụng, dễ mở rộng, keras giúp người dùng xây dựng các deep learning model một cách đơn giản.

MNIST Dataset

- Xây dựng bài toán
 - Bạn có ảnh xám kích thước 28×28 của chữ số từ 1 đến 9 và bạn muốn dự đoán số đây là số mấy. Ví dụ:

Dữ liệu đầu tiên trong MNIST dataset



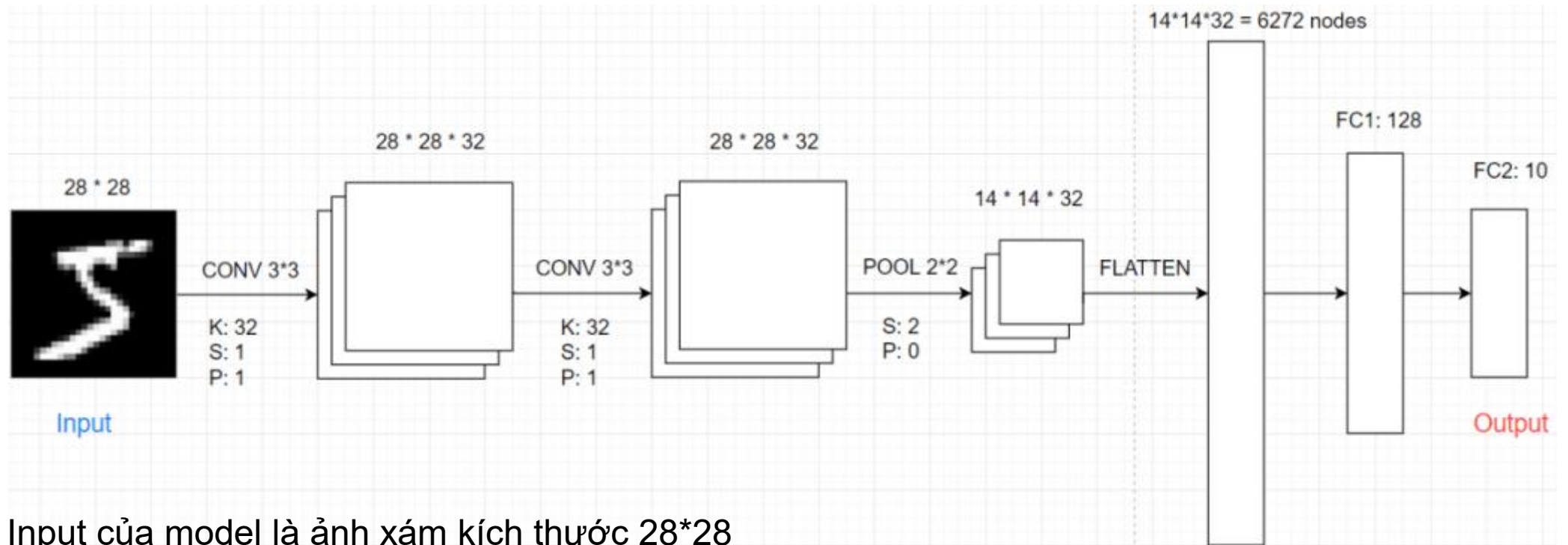
MNIST Dataset

- Chuẩn bị dữ liệu
 - MNIST là bộ cơ sở dữ liệu về chữ số viết tay, bao gồm 2 tập con: tập huấn luyện (training set) gồm 60.000 ảnh các chữ số viết tay và tập kiểm tra (test set) gồm 10.000 ảnh các chữ số.

MNIST dataset có 60.000 dữ liệu ở training set ở trong MNIST, ta sẽ chia ra 50.000 dữ liệu cho training set và 10.000 dữ liệu cho validation set. Vẫn giữ nguyên 10.000 dữ liệu của test set.

MNIST Dataset

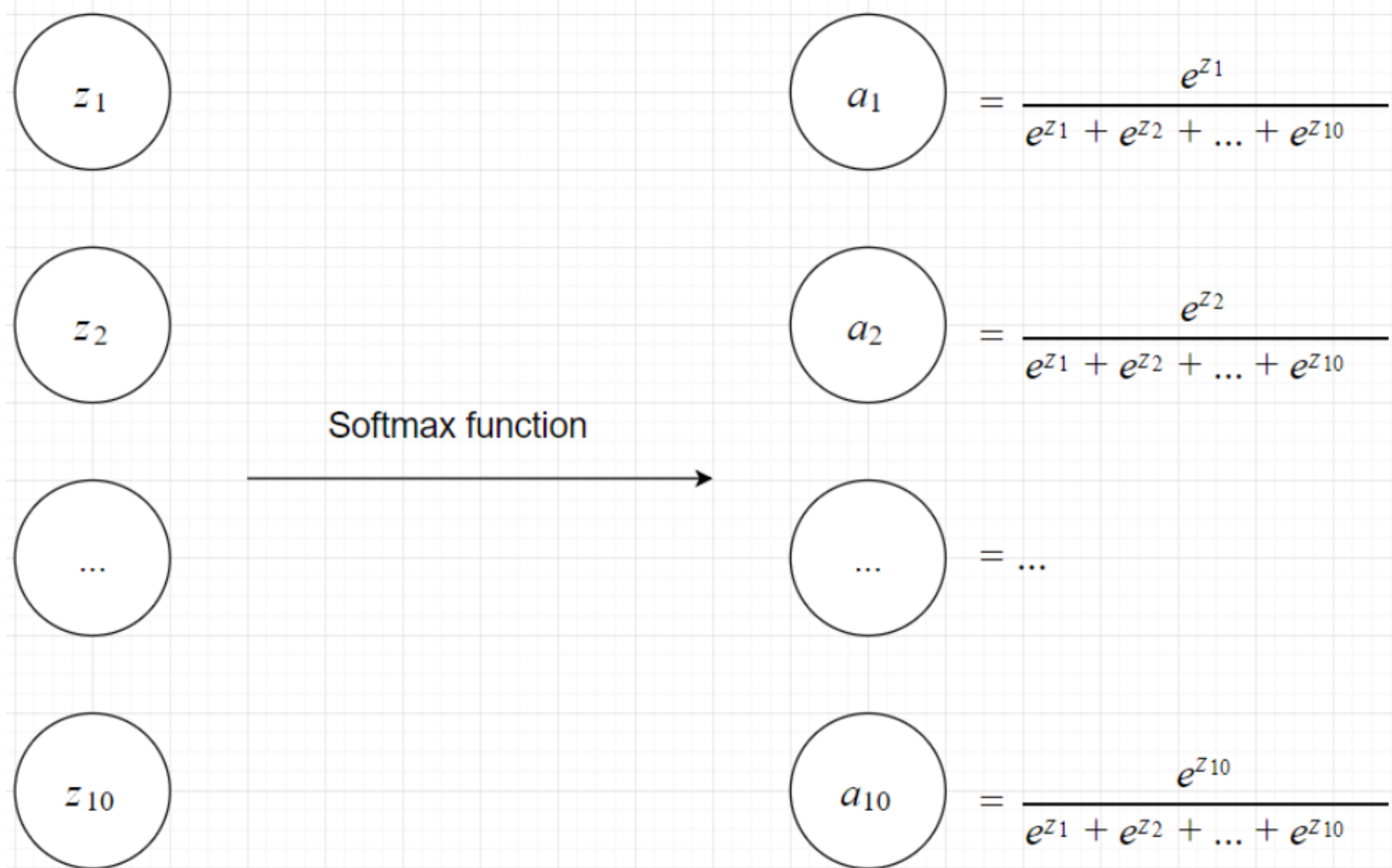
- Xây dựng model
 - Vì input của model là ảnh nên nghĩ ngay đến CNN.
 - Mô hình chung bài toán CNN: Input image -> Convolutional layer (Conv) + Pooling layer (Pool) -> Fully connected layer (FC) -> Output.



MNIST Dataset

- Xây dựng model
 - Softmax function
 - Giống như bài logistic regression, thay vì chỉ muốn kết quả là ảnh là số mấy, ta muốn dự đoán phần trăm của ảnh là số nào. Ví dụ: 90% ảnh là số 5, 1% ảnh là số 1, ...
 - Do sau bước tính tổng linear cho ra các giá trị thực nên cần dùng softmax function dùng để chuyển đổi giá trị thực trong các node ở output layer sang giá trị phần trăm.
 - Vì mỗi ảnh sẽ thuộc 1 class từ 0 đến 9, nên tất cả sẽ có 10 class. Nên output layer sẽ có 10 node để tương ứng với phần trăm ảnh là số 0,1, ..., 9.
 - Ví dụ: a_6 là xác suất ảnh là số 5. (Sự khác biệt chỉ số do các số bắt đầu từ 0 trong khi chỉ số của node trong layer bắt đầu từ 1)

MNIST Dataset



MNIST Dataset

- Xây dựng model
 - Softmax function

Tổng quát sau hàm activation: $a_k = \frac{e^{z_k}}{\sum_{i=1}^{10} e^{z_i}}$.

Nhận xét:

- $\sum_{i=1}^{10} a_i = 1$
- $0 < a_i < 1$

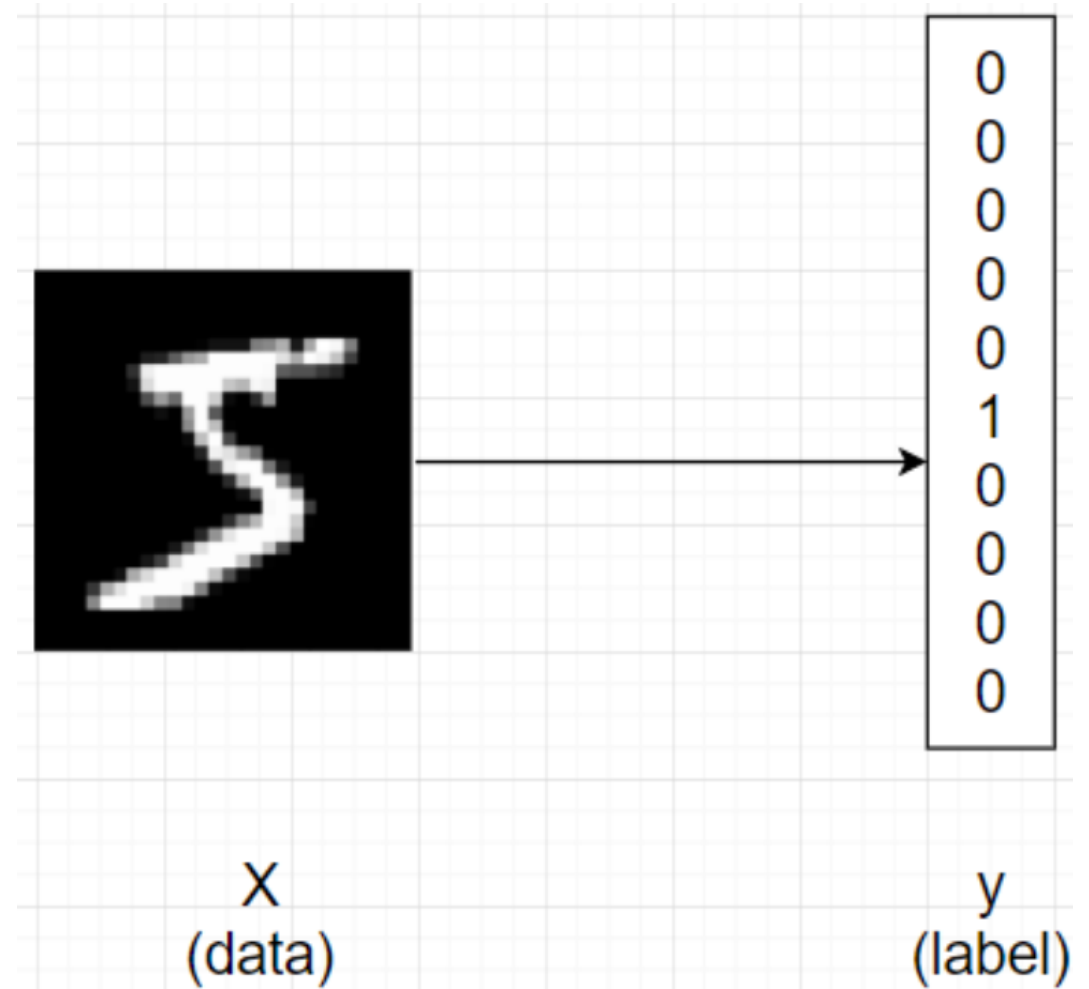
Do đó ta có thể coi a_i là xác suất ảnh là số (i-1).

MNIST Dataset

- Xây dựng model
 - Softmax function
 - Với các bài toán classification (phân loại), nếu có 2 lớp thì hàm activation ở output layer là hàm sigmoid và hàm loss function là `binary_crossentropy`, còn nhiều hơn 2 lớp thì hàm activation ở output layer là hàm softmax với loss function là hàm `categorical_crossentropy`
- => Output layer có 10 nodes và activation là softmax function.**

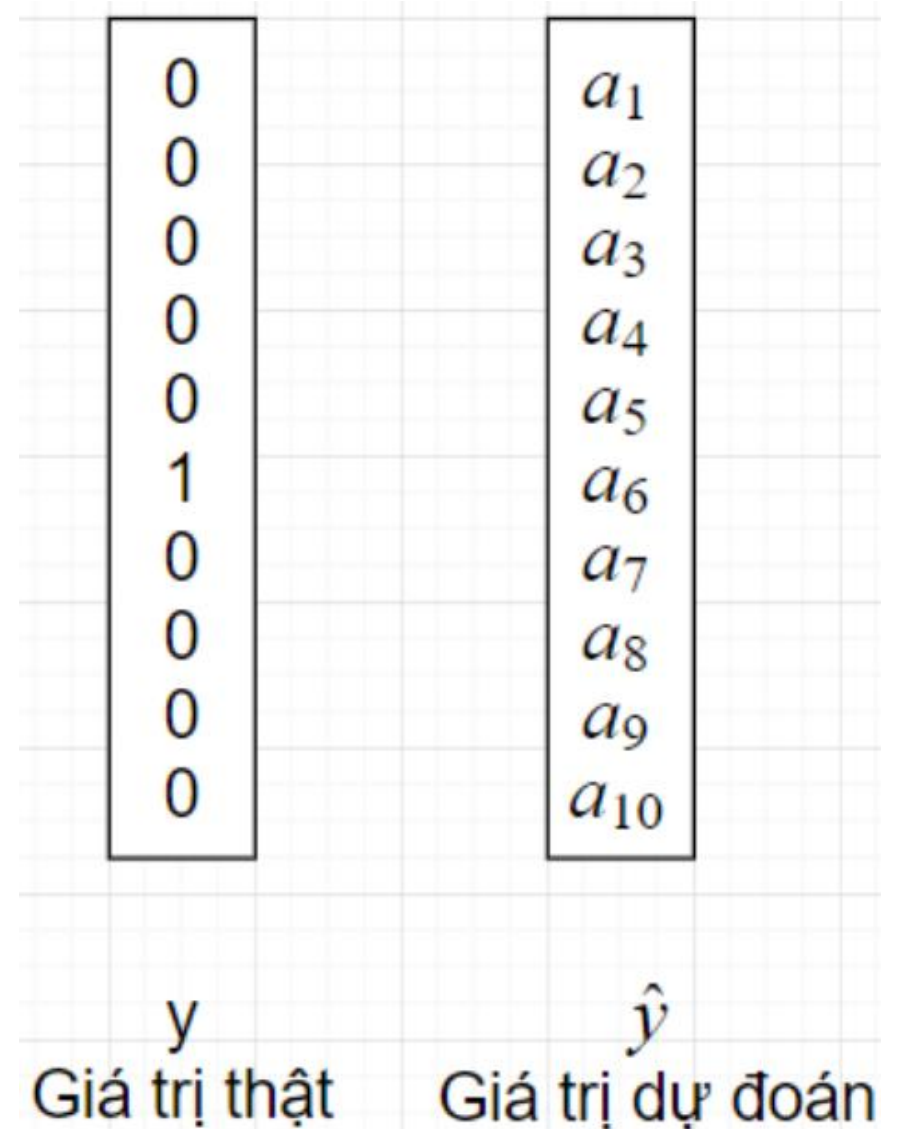
MNIST Dataset

- Xây dựng model
 - Loss function
 - Chuyển đổi các nhãn thành dạng nhãn phân loại (one-hot encoding).
→ chuyển đổi label của ảnh từ giá trị số sang vector cùng kích thước với output của model
 - Ở hình bên, để ý là nhãn (label) của data là số i thì vector v kích thước 10×1 có $v_{i+1} = 1$ và các giá trị khác bằng 0. So với quy ước về phần trăm ở trên thì one-hot encoding có ý nghĩa là ta chắc chắn 100% ảnh này là số 5.



MNIST Dataset

- Xây dựng model
 - Loss function
 - Giờ ta có giá trị thật (label) dạng one-hot encoding giá trị dự đoán ở output layer sau hàm softmax function cùng kích thước 10×1 .
 - Ta cần định nghĩa hàm loss function để đánh giá độ tốt của model.
 - Ví dụ bên, mong muốn là a_6 gần 1 còn các giá trị a khác gần 0 thì model dự đoán đúng được ảnh đầu vào là ảnh số 5.



MNIST Dataset

- Xây dựng model

- Loss function

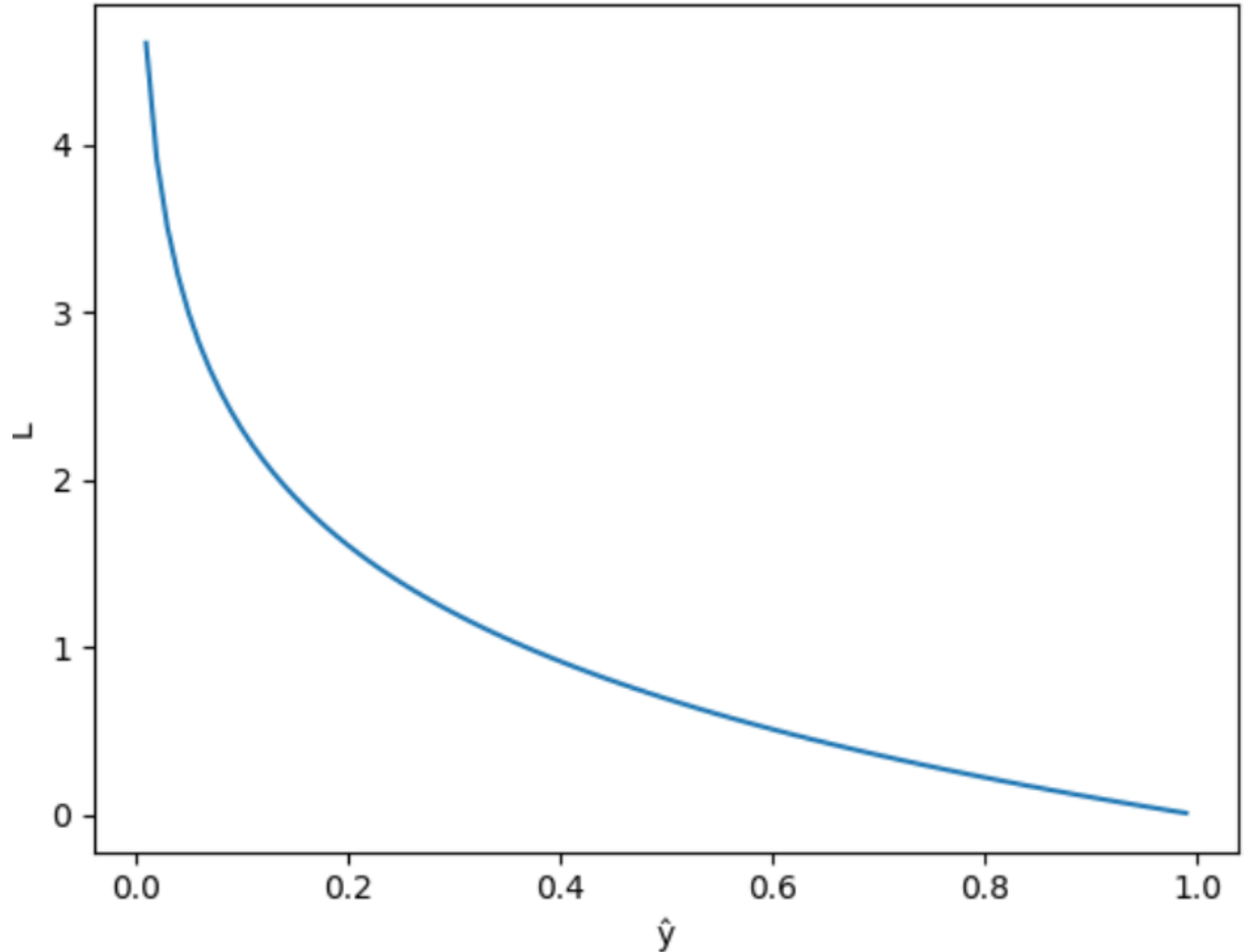
- Ta định nghĩa loss function: $L = - \sum_{i=1}^{10} y_i * \log(\hat{y}_i)$

- Giả sử ảnh là số 5 thì $L = -\log(\hat{y}_6)$

- Hàm loss function này trong keras gọi là "categorical_crossentropy"

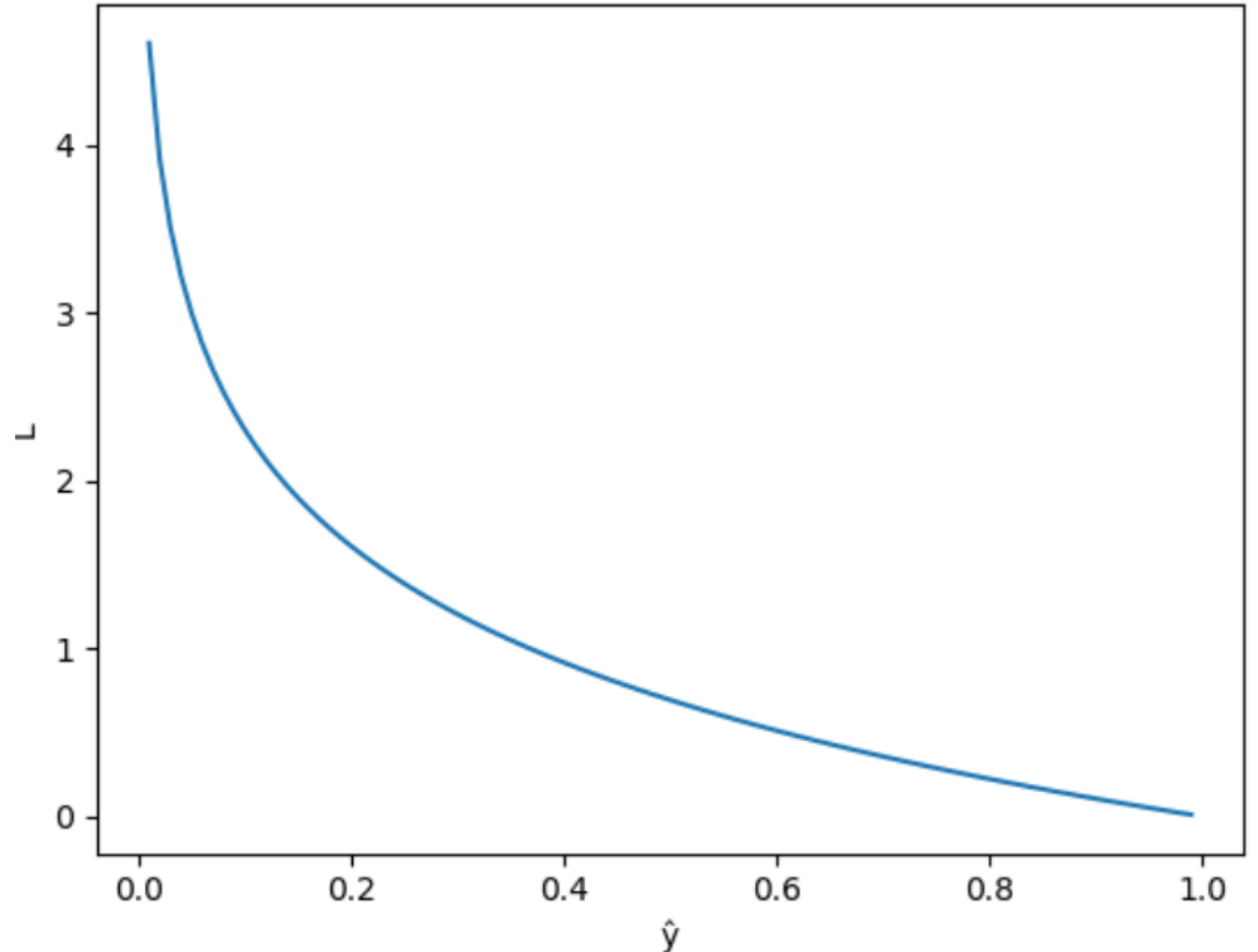
MNIST Dataset

- Xây dựng model
 - Loss function
 - Hàm L giảm dần từ 0 đến 1
 - Khi model dự đoán $\hat{y}_6$ gần 1, tức giá trị dự đoán gần với giá trị thật y_6 thì L nhỏ, xấp xỉ 0
 - Khi model dự đoán $\hat{y}_6$ gần 0, tức giá trị dự đoán ngược lại giá trị thật y_6 thì L rất lớn



MNIST Dataset

- Xây dựng model
 - Loss function
 - Hàm L nhỏ khi giá trị model dự đoán gần với giá trị thật và rất lớn khi model dự đoán sai, hay nói cách khác L càng nhỏ thì model dự đoán càng gần với giá trị thật. => Bài toán tìm model trở thành tìm giá trị nhỏ nhất của L .



MNIST Dataset

- Ứng dụng của việc phân loại ảnh
 - Chuẩn đoán ảnh X-ray của bệnh nhân có bị ung thư hay không
 - Phân loại, nhận diện được các chữ, số viết tay => tự động đọc được biển số xe, văn bản.
 - Phân loại được các biển báo giao thông => hỗ trợ cho ô tô tự lái

MNIST Dataset

- Mô hình keras
 - Sequential là một mô hình tuyến tính gồm các lớp kế tiếp nhau, phù hợp cho các mô hình đơn giản. Các bước sử dụng Sequential:
 1. Cài đặt Keras và các thư viện liên quan: `pip install keras tensorflow`
 2. Import các module cần thiết:

```
from keras.models import Sequential
from keras.layers import Dense
```
 3. Tạo mô hình Sequential: `model = Sequential()`
 4. Thêm các lớp vào mô hình:

```
model.add(Dense(units=64, activation='relu', input_dim=100))
model.add(Dense(units=10, activation='softmax'))
```

MNIST Dataset

- Mô hình keras

- Các bước sử dụng Sequential:

- 5. Biên dịch mô hình:

- ```
model.compile(loss='categorical_crossentropy',
 optimizer='sgd',
 metrics=['accuracy'])
```

- 6. Huấn luyện mô hình:

- ```
model.fit(x_train, y_train, epochs=5, batch_size=32)
```

MNIST Dataset

- Mô hình keras
 - Khi bạn huấn luyện một mô hình Sequential trong Keras, kết quả huấn luyện sẽ được lưu trữ trong một đối tượng History. Đối tượng này chứa thông tin về quá trình huấn luyện, bao gồm các giá trị mất mát (loss), độ chính xác (accuracy), và các số liệu khác cho mỗi epoch.
 - Các key chính trong [history.history](#):
 - loss: Mất mát trên tập huấn luyện.
 - val_loss: Mất mát trên tập kiểm tra (validation).
 - accuracy hoặc acc: Độ chính xác trên tập huấn luyện.
 - val_accuracy hoặc val_acc: Độ chính xác trên tập kiểm tra (validation).
 - Các số liệu tùy chỉnh khác nếu thêm chúng vào khi biên dịch mô hình.

MNIST Dataset

- Mô hình keras
 - Conv2D và MaxPooling2D là hai lớp cơ bản để xây dựng mạng nơ-ron tích chập (Convolutional Neural Networks - CNNs) cho xử lý hình ảnh và nhận dạng mẫu.
 - Conv2D (Convolutional Layer 2D):
 - Lớp Conv2D thực hiện phép tích chập 2D trên các input, thường là hình ảnh.
 - Nó sử dụng các kernel (bộ lọc) để trích xuất các đặc trưng (features) từ ảnh.
 - Cú pháp:

```
from keras.layers import Conv2D
model.add(Conv2D(filters=32, kernel_size=(3, 3), activation='relu',
input_shape=(64, 64, 3)))
```

 - filters: Số lượng bộ lọc (kernel) được áp dụng.
 - kernel_size: Kích thước của kernel, thường là (3, 3) hoặc (5, 5).
 - activation: Hàm kích hoạt, ví dụ: 'relu', 'sigmoid'.
 - input_shape: Kích thước của input (chiều cao, chiều rộng, số kênh màu).

MNIST Dataset

- Mô hình keras
 - MaxPooling2D (Max Pooling Layer 2D):
 - Lớp MaxPooling2D thực hiện phép gộp tối đa (max pooling) trên các input.
 - Mục đích là giảm kích thước của các feature map, giảm số lượng tham số và tính toán, và ngăn chặn overfitting.
 - Cú pháp:

```
from keras.layers import MaxPooling2D  
model.add(MaxPooling2D(pool_size=(2, 2)))
```

 - pool_size: Kích thước của bộ lọc pooling, thường là (2, 2).

MNIST Dataset

- Mô hình keras
 - Các lớp Dense, Dropout, Activation và Flatten là những lớp cơ bản và thường được sử dụng trong Keras để xây dựng các mô hình học sâu.
 - Lớp Dense:
 - Đây là lớp kết nối đầy đủ (fully connected layer) trong mạng nơ-ron.
 - Mỗi nơ-ron trong lớp này đều kết nối với tất cả các nơ-ron trong lớp trước đó.
 - Cú pháp:

```
from keras.layers import Dense
```

```
model.add(Dense(units=64, activation='relu', input_dim=100))
```

 - units: số lượng nơ-ron trong lớp này.
 - activation là hàm kích hoạt. Ví dụ: 'relu', 'sigmoid', 'tanh'.
 - input_dim là số chiều của dữ liệu đầu vào.

MNIST Dataset

- Mô hình keras
 - Các lớp Dense, Dropout, Activation và Flatten là những lớp cơ bản và thường được sử dụng trong Keras để xây dựng các mô hình học sâu.
 - Lớp Dropout:
 - Lớp này được sử dụng để ngăn chặn overfitting.
 - Một số nơ-ron sẽ bị "bỏ qua" ngẫu nhiên trong quá trình huấn luyện.
 - Cú pháp:

```
from keras.layers import Dropout  
model.add(Dropout(rate=0.5))
```

 - rate là tỷ lệ nơ-ron bị bỏ qua. Ở đây, 50% nơ-ron sẽ bị bỏ qua.

MNIST Dataset

- Mô hình keras
 - Các lớp Dense, Dropout, Activation và Flatten là những lớp cơ bản và thường được sử dụng trong Keras để xây dựng các mô hình học sâu.
 - Lớp Activation:
 - Lớp này áp dụng hàm kích hoạt cho lớp trước đó.
 - Có thể thêm lớp Activation trực tiếp hoặc truyền hàm kích hoạt vào lớp khác (như Dense).
 - Cú pháp:

```
from keras.layers import Activation  
  
model.add(Dense(units=64, input_dim=100))  
  
model.add(Activation('relu'))
```

 - Các hàm kích hoạt phổ biến: 'relu', 'sigmoid', 'softmax', 'tanh'.

MNIST Dataset

- Mô hình keras
 - Các lớp Dense, Dropout, Activation và Flatten là những lớp cơ bản và thường được sử dụng trong Keras để xây dựng các mô hình học sâu.
 - Lớp Flatten:
 - Lớp này chuyển đổi dữ liệu đầu vào từ ma trận hoặc tensor thành một vector phẳng.
 - Thường được sử dụng khi chuyển từ lớp Convolutional sang lớp Dense.
 - Cú pháp:

```
from keras.layers import Flatten  
  
model.add(Flatten())
```

MNIST Dataset

- Mô hình keras
 - Thư viện `keras.api.utils`: cung cấp các tiện ích hỗ trợ cho việc xây dựng và huấn luyện mô hình học sâu.
 - Cách sử dụng: `from keras.api import utils`
 - `get_file`: Tải xuống và lưu trữ các tệp từ URL.
`file_path = utils.get_file('filename', 'https://example.com/filepath')`
 - `to_categorical`: Chuyển đổi các nhãn thành dạng nhãn phân loại (one-hot encoding).
`y_binary = utils.to_categorical(y, num_classes=10)`
 - `plot_model`: Vẽ biểu đồ mô hình.
`utils.plot_model(model, to_file='model.png', show_shapes=True)`